Midterm Report — Reproducing Dreamer V1 from Pixels

CMU 11-785 | Preparation and bounded validation

Abstract

This project investigates whether the original Dreamer V1 agent can be reproduced from pixel observations in the DeepMind Control Suite. The immediate target is `cartpole_swingup`; `walker_walk` and `ball_in_cup_catch` are reserved for later experiments. The project is deliberately anchored to the authors' original TensorFlow 1 implementation rather than a newer Dreamer variant. At the midterm point, the work has completed the source audit, experiment protocol, repository bootstrap, local environment checks, and a bounded GPU-backed pixel-environment smoke test. It has not yet completed Dreamer learning or claimed a paper reproduction.

The main empirical result available today is an environment and compute preflight on the CMU-verified Kaggle account. A Tesla T4 session successfully ran PyTorch CUDA, installed `dm-control` with an EGL renderer, created `cartpole_swingup` pixel observations of shape 64 x 64 x 3, and executed two independent 1,000-environment-step random-policy rollouts. With action repeat 2, each rollout contained 500 decision steps. The observed episodic returns were 81.916 and 35.688. These values are random-policy sanity checks, not learned-policy results. They establish that a small GPU budget is sufficient for integration testing, while the formal Dreamer reproduction still requires a stable Linux GPU workflow and a much larger training budget.

1. Problem, motivation, and research target

Dreamer learns a latent world model from image observations and improves behavior by imagining future latent trajectories. This makes it a useful test case for a project centered on reinforcement learning from pixels: the input is high-dimensional, the control objective is continuous, and the agent must learn both representation and dynamics. The project asks a practical reproduction question: can the central Dreamer V1 pipeline be reconstructed and evaluated under a documented environment contract, before adding a research extension?

The first target is `cartpole_swingup` because it is small enough for debugging while still requiring visually grounded control. The two follow-up tasks are `walker_walk` and `ball_in_cup_catch`. The project will report episodic return and learning curves. Classification accuracy is not an appropriate metric for this control problem.

The target paper is Hafner et al., *Dream to Control: Learning Behaviors by Latent Imagination* (2019). The chosen code reference is the authors' `google-research/dreamer` repository at commit `d517542f9777c53864ff6f2683cea88be9ea98ac`. This choice keeps the reproduction tied to Dreamer V1 and makes the main correctness risk explicit: the code is historically old and depends on a TensorFlow 1-era software stack. The maintained TensorFlow 2 repository at `danijar/dreamer` is useful as a comparison source, but it is not silently substituted for the V1 target.

2. Related work and implementation audit

The paper describes a recurrent state-space model (RSSM) that separates a deterministic recurrent state from a stochastic latent state. An encoder maps pixels to embeddings. The world model predicts rewards, continuation, and future latent states. The actor is optimized through imagined trajectories, and the critic estimates imagined returns. The replay buffer stores sequences of real interaction, rather than isolated images, so that the recurrent model can learn temporal structure.

The repository audit distinguishes three layers. First, the paper is the source for the algorithmic target and reported evaluation protocol. Second, the pinned author repository is the source for the concrete V1 code path and default experiment configuration. Third, the local project adds documentation, configuration records, tests, and bounded smoke-test tooling. No new learning algorithm has been inserted into the upstream implementation.

The paper and repository are not identical in every operational detail. The repository contains defaults for task selection, sequence length, image size, action repeat, optimization schedules, logging, and checkpointing. Some of those values are represented in code or configuration rather than stated in one concise table in the paper. Therefore, the local protocol marks any value that still needs confirmation as `TBD` instead of inventing a paper number. The local project also separates environment steps from decision steps: with action repeat 2, 1,000 environment steps correspond to 500 policy decisions.

The original repository's license and attribution remain upstream-owned. The local repository does not relicense copied upstream source. The project files identify the upstream URL and commit so that the exact code can be retrieved again. The newer `danijar/dreamer` repository has different maintenance and dependency assumptions and is therefore treated as a reference, not as the selected implementation.

3. Methodology and reproduction protocol

Environment and observation contract

The first environment is DeepMind Control Suite `cartpole_swingup`. The initial pixel wrapper uses 64 x 64 RGB observations. The action space is one-dimensional and bounded by the environment action specification. The current smoke protocol uses action repeat 2. The environment step count is the number of calls to `env.step`; the decision count is the number of sampled actions. Episodic return is the sum of environment rewards within an episode, with no normalization applied in the smoke test.

The formal run will preserve the observation key, pixel shape, action bounds, action repeat, seed, environment-step budget, checkpoint directory, and evaluation episode count in an experiment record. A run is not considered comparable if those fields are changed without being recorded.

Training and evaluation plan

The primary comparison is the paper's continuous-control result for the selected task, with learning curves as the central artifact. The project will first reproduce a single-task baseline, then extend to additional tasks only after the baseline pipeline is stable. The intended progression is:

1. Import and renderer preflight on Linux GPU.
2. Very short Dreamer debug run with checkpoint creation and restore.
3. Short `cartpole_swingup` training run to catch numerical, replay, and logging errors.
4. Formal training budget with multiple random seeds.
5. Deterministic evaluation episodes and a return-versus-environment-steps plot.

The exact formal training budget, seed count, and evaluation episode count remain to be confirmed against the pinned configuration and cluster limits. The project will not label a short smoke test as a reproduction. It will report mean return, variability across seeds, and the environment-step axis used to produce each curve.

Baselines and planned iteration

The first baseline is a random policy. Its purpose is not to be competitive; it validates the environment contract and provides a lower-complexity reference for debugging. The next baseline is the selected Dreamer V1 implementation with the pinned configuration. A later extension may test an action-repeat change or a representation/preprocessing ablation, but no extension will be interpreted until the baseline has a working checkpoint and evaluation path.

The midterm rubric asks for evidence of iteration. The two random-policy seeds reported below are the first controlled configuration check. The next iteration is a change from CPU/macOS inspection to a Linux EGL-backed Kaggle environment, because the local Mac did not provide a usable MuJoCo OpenGL backend. This is an infrastructure adjustment, not an algorithmic improvement.

4. Compute feasibility and bounded experiments

The local Mac is useful for documentation, configuration, tests, and static interface checks. It is not currently a reliable pixel-training environment: the project-local `dm-control` installation could be prepared, but MuJoCo rendering could not find a usable OSMesa or EGL/OpenGL backend under the bounded local check. TensorFlow 1.13.1 and TensorFlow Probability 0.6.0 were not installed locally. No long CPU or MPS training was started.

The CMU Kaggle account provided a useful, bounded alternative. The notebook used a Tesla T4 and PyTorch `2.11.0+cu128`. A tiny 1,000-step synthetic CUDA forward/backward check completed in 1.149 seconds. The standard `pip install dm-control` path attempted to build `labmaze` and failed because the environment did not provide the required Bazel build path. The notebook then used `dm-control --no-deps` and installed the minimal runtime packages separately. The first renderer attempt failed because GLFW required an X11 display. Setting `MUJOCO_GL=egl` allowed headless rendering.

The resulting pixel smoke test used `cartpole_swingup`, a 64 x 64 x 3 observation, action repeat 2, and a 1,000-environment-step bound. It saved a short GIF and a JSON summary inside Kaggle's working directory. The first run used random seed 0 and obtained episodic return 81.915742. The second used random seed 1 and obtained episodic return 35.688017. Both runs completed 500 decision steps. The GPU session was stopped immediately after the bounded tests.

Run	Policy	Seed	Environment steps	Decision steps	Pixel shape	Action repeat	Episodic return	Status
1	Random	0	1,000	500	64 x 64 x 3	2	81.915742	Passed
2	Random	1	1,000	500	64 x 64 x 3	2	35.688017	Passed

These runs provide evidence for the data path and step accounting only. They do not demonstrate that the model learns, that the actor improves behavior, or that the paper's curve has been reproduced. They also reveal a reproducibility issue that must be recorded: Kaggle's current image requires a headless EGL setup and a dependency workaround for `labmaze`. The university Linux environment may resolve that issue differently.

Kaggle is therefore sufficient for small smoke tests, short model checks, and possibly a toy training experiment. It is not yet the preferred place for a formal reproduction because sessions, package versions, quota limits, and persistent storage are less controlled than a university allocation. The project has also identified an active ACCESS allocation with approximately 3,000 GPU hours, but the remote workflow has not been executed and no scheduler has been assumed.

5. Results, failure analysis, and limitations

Results available at midterm

Passed checks include project metadata tests, Git whitespace validation, isolated-environment setup, the synthetic CUDA forward/backward check, headless MuJoCo pixel rendering on Kaggle, two bounded random rollouts, and short GIF generation. These results are reproducible at the notebook level through the CMU-verified Kaggle account and the saved draft notebook URL recorded in the project handoff.

The local macOS pixel probe failed because the available renderer backend was insufficient. The normal `dm-control` installation also encountered a `labmaze` build problem in Kaggle. These are meaningful failure cases rather than missing results: they identify environment assumptions that must be handled in the final Linux setup. The first non-CMU Kaggle notebook session also returned an authentication/internal error, so that session is not used as evidence. The correct CMU account subsequently produced a working T4 session.

What remains unverified

The following claims are intentionally not made: no Dreamer V1 policy has been trained, no learned episodic-return curve exists, no checkpoint has been restored after training, no comparison against the paper's numerical result has been completed, and no claim is made about GPU-hours consumed by the formal experiment. The compatibility of the pinned TensorFlow 1 stack with a current Linux CUDA image remains to be tested. The exact university login host, storage policy, and scheduler interface remain to be confirmed.

The most important scientific limitation is that the available results are infrastructure results. Random-policy return varies substantially across seeds and is not a quality benchmark. The correct next evidence is a short learned baseline with a visible loss/return trace, a successful checkpoint restore, and a fixed evaluation protocol.

6. Next steps

The next executable task is a Linux GPU preflight for the pinned Dreamer V1 code: verify Python and TensorFlow compatibility, import the selected modules, instantiate the environment, run a short debug budget, write a checkpoint, restore it, and evaluate a small number of episodes. Only after that preflight passes should the project spend the larger formal budget. The university should confirm the SSH host and username, VPN or jump-host requirements, whether the login node is separate from compute nodes, whether a scheduler is used, account and partition names if Slurm is used, CUDA and driver versions, Python/Conda/module or container policy, project and scratch paths, and storage quotas.

The report's immediate deliverable is a transparent midterm state: the repository is organized, the source choice is fixed, the local and Kaggle failures are documented, the environment contract is tested, and the remaining uncertainty is narrowed to Linux compatibility and learned training. This is sufficient evidence of progress without presenting fabricated curves or example performance numbers.

References

1. Hafner et al. *Dream to Control: Learning Behaviors by Latent Imagination*. arXiv:1912.01603. https://arxiv.org/abs/1912.01603
2. Google Research. Original Dreamer V1 implementation. https://github.com/google-research/dreamer
3. Danijar. TensorFlow 2 Dreamer reference implementation. https://github.com/danijar/dreamer
4. Google DeepMind. DeepMind Control Suite. https://github.com/google-deepmind/dm_control
5. Kaggle. Notebooks documentation. https://www.kaggle.com/docs/notebooks
6. Kaggle. Efficient GPU usage. https://www.kaggle.com/docs/efficient-gpu-usage